

You are strongly recommended to explore your own ideas!!

Application Project

For the application project, we provide five application domains for your reference. But you are strongly encouraged to expand beyond the provided dataset. Your goal will be to propose an application that requires machine learning using the dataset. The novelty lies in the pipeline that you develop, and you will be focusing on the various aspects of developing a complete machine learning solution to automate a real-life need. This will include, formulating a task, selecting data, feature engineering, determining the best features for your model, implementing a model, determining hyperparameters, testing your model for problematic behavior, and evaluating using the right metrics that the stakeholders in the applications care. You can use any software library for this task. The way you process your data, and the choice of models will depend on the domain that you're working on. You will be judged based on the work you put into the project beyond the available software libraries.

There are some general machine learning dataset repositories:

1. UCI machine learning repository: <https://archive.ics.uci.edu/>, great for basic classification and regression problems
2. [Data.gov](https://data.gov/) (<https://data.gov/>) Good to find high-quality open data from the government
3. Awesome public datasets git repo: <https://github.com/awesomedata/awesome-public-datasets>, starred by many
4. Kaggle datasets: <https://www.kaggle.com/datasets>
5. OpenML: <https://www.openml.org/>
6. Huggingface (<https://huggingface.co/papers/trending>) Good to find datasets and codes used by recent papers
7. For more “niche” applications, NeurIPS has a benchmark track. Papers there usually provide a git repo to host their benchmark data. For example, this is the proceeding page for last year: <https://nips.cc/Conferences/2021/DatasetsBenchmarks/AcceptedPapers>

You can also select a dataset from one of the following domains.

8. Medical informatics and Genetics
 - 10x Genomics <https://www.10xgenomics.com/resources/datasets>
 - UCSC Genome Browser <https://genome.ucsc.edu/>
 - Gencode <https://www.encodegenes.org/>
 - Protein Classification <http://scop.mrc-lmb.cam.ac.uk>
9. Healthcare delivery and public health
 - COVID cases and deaths: <https://ourworldindata.org/covid-deaths>
 - WHO Statistics: <https://apps.who.int/gho/data/node.resources>
 - CDC Statistics: <https://data.cdc.gov/browse>

10. Textual analysis

- Wikipedia articles: https://en.wikipedia.org/wiki/Wikipedia:Database_download
- Sentiment prediction: <http://archive.ics.uci.edu/ml/datasets/Sentiment+Labelled+Sentences>
- 20newsgroups: https://scikit-learn.org/0.19/datasets/twenty_newsgroups.html
- Translation of Canadian Parliament: <https://www.isi.edu/natural-language/download/hansard/>
- Hate speech on Twitter <https://data.world/thomasrdavidson/hate-speech-and-offensive-language>

11. Image analysis

- Facial recognition: <http://vision.ucsd.edu/content/yale-face-database>
- Art objects (paintings, sculptures, drawings etc) from the Rijksmuseum museum in Amsterdam: https://figshare.com/articles/Rijksmuseum_Challenge_2014/5660617
- Satellite Imagery for Natural Disasters: <https://www.digitalglobe.com/ecosystem/open-data>

12. Finance and economics

- World Bank Open Data: <https://data.worldbank.org/>
- International Monetary Fund Data: <https://www.imf.org/en/Data>
- Quandl: <https://www.quandl.com/> (Make sure to select free data)
- Stock prices from [pandas_datareader](#)

13. Environmental Applications: AI for Earth

- <https://microsoft.github.io/AIforEarthDataSets/>

14. AI for social good datasets:

- Check the many good datasets here: <https://github.com/shreyashankar/datasets-for-good>

Methods Projects

We have learned about several algorithms and the assumptions behind them. We also commented that these observations are not valid in real data. For example, the assumption of independent and identically distributed observations does not hold for many data. Further, it is possible that there are behaviors other than high prediction accuracy that a real world application might need. For example, you want your classifier to be fair to individuals who belong to a minority group. Real data also requires coming up with smart ways to deal with noise, missingness and unknown confounders. Additionally, you might care about other model requirements like interpretability: why did the model predict a certain outcome?

We suggest the following topics for the methods projects. But you are strongly encouraged to define your own desired properties of the ML algorithms, based on what you have learned. Your task will be to evaluate a method and propose/modify a method, such as creating a new loss function, constraints, data preprocessing, etc. with the goal of solving one of the tasks. You will demonstrate that your model improved on the desired task, and can be used to achieve your

stated goal, on selected datasets, either simulated or real data depending on your project needs. You **must** evaluate your method on one of the simple datasets contained within sklearn.datasets, if applicable, but you are encouraged to also use a real-world dataset to explore a more interesting application. The exact evaluation will depend on your proposed method. You do not need to implement the methods from scratch; you will be allowed to use existing software libraries. However, we will judge you based on the amount you do beyond existing implementations or any potential coding help you have access to. Additionally, you should compare your method to an existing method, such as a classifier from Scikit Learn.

15. Fairness in Machine Learning

- A project that demonstrates fairness aspects of a model or dataset, and proposes methods to improve fairness in a measurable way
- One example project is the famous gender shades project: <http://gendershades.org/overview.html>, which investigated the performance difference in the gender classification.

16. Interpretable Machine Learning

- Methods for interpreting trained ML models
- Useful references: <https://christophm.github.io/interpretable-ml-book/neural-networks.html>

17. Adversarial attack and defense

Implement a deep learning based classification, one adversarial example attack and also one defense mechanism for the attack.

Supporting materials need to read before project:

- MNIST data classify model (you may start with MNIST) <https://github.com/pytorch/examples/tree/main/mnist>
- Attack mechanism:
Fast Gradient Sign Method (FGSM): a method to add the noise (not random noise) whose direction is the same as the gradient of the cost function with respect to the data. <https://arxiv.org/abs/1412.6572>
<https://pyimagesearch.com/2021/03/01/adversarial-attacks-with-fgsm-fast-gradient-sign-method/>
- Defense mechanism:
Defensive Distillation: <https://arxiv.org/abs/1511.04508>

Recommend workflow (using MNIST as an example):

- Step 1: Implement the pytorch example model for MNIST
 - Plot the training and validation loss
- Step 2: Applied the attack method on the model
 - Plot accuracy when set noise magnitude Epsilon to 0, 0.01, 0.02, 0.05, 0.1 and 0.2 (Epsilon is a parameter you will learn from the supporting materials)
 - Visualize sample adversarial examples for different Epsilon
- Step 3: Applied the defense method on the model

- Plot accuracy when set Epsilon to 0, 0.01, 0.02, 0.05, 0.1 and 0.2
- Visualize sample adversarial examples for different Epsilon

11. Domain adaptation/generalization using one or multiple WILDS data

WILDS is a benchmark of in-the-wild distribution shifts spanning diverse data modalities and applications, from tumor identification to wildlife monitoring to poverty mapping.

Data link: <https://wilds.stanford.edu/>

Paper link: <https://arxiv.org/abs/2012.07421>

Pick one or multiple datasets from the WILDS, implement one of the domain adaptation or generalization methods (for example, one of the methods listed here

<https://github.com/facebookresearch/DomainBed>). Train the model in the (multiple) source distributions and test in the target distribution. Analyze the performance in different domains.

Besides accuracy, evaluate the uncertainty estimates as well. Do they work well in your chosen data or domain? How can you improve them?